SurakshaX

Women's safety app · Technical report

Tech stack · Features · APIs · Backend · Database · ML models

Version	App 3.1.0 · API 3.0.0
Date	23 October 2026
Repository	github.com/Jotthecode/Suraksha_X (branch main)
Live API	https://surakshax-api.onrender.com
Platforms	Android (APK), web preview; API on Render

Contents

1. Summary
2. System architecture
3. Tech stack
4. Features
5. How the backend works
6. API reference
7. Database schema
8. Machine-learning models
9. Rule-based detection on the phone
10. External services and data
11. Security and privacy
12. Deployment and configuration
13. Limitations and next steps

1. Summary

SurakshaX is a personal-safety app for women in India. One tap, a spoken distress word, a secret code phrase, a hard phone shake or sudden running starts an SOS. The phone texts every emergency contact from its own SIM with a Google Maps link and a live-tracking link, calls the first contact, and reports the incident to the server, which finds the nearest police stations, can send Twilio SMS and a voice call, and shows the alert on a password-protected police dashboard. Guardians follow the live location in any browser without installing anything.

The app also scores how safe an area is (Random Forest model over OpenStreetMap features), compares safe and short walking routes on real roads, suggests places as you type, lists nearby police stations, and can keep protecting in the background through an Android foreground service.

- **Mobile:** Expo SDK 57 / React Native 0.86 Android app with a small custom Kotlin module.
- **Backend:** Python 3.12, FastAPI, SQLAlchemy, PostgreSQL on Render (SQLite locally).
- **ML:** a 10-class voice CNN (ONNX Runtime) and a 3-class crime-risk Random Forest (scikit-learn).
- **Maps & data:** OpenStreetMap ecosystem (OSRM, Photon, Nominatim, map tiles) plus a bundled India-wide grid of street lamps and shops/amenities and 4,370 police stations. No paid map API key.

> **Honesty notes.** Both ML models are trained on synthetic data (text-to-speech voices; heuristic risk labels), so real-world accuracy is lower than the test numbers in this report. The voice trigger and area-risk score are aids, never a replacement for the SOS button or calling 112.

2. System architecture

Three tiers: the Android app, the SurakshaX API on Render with its PostgreSQL database, and external map and messaging services. Guardians and police use the same API through plain web pages.

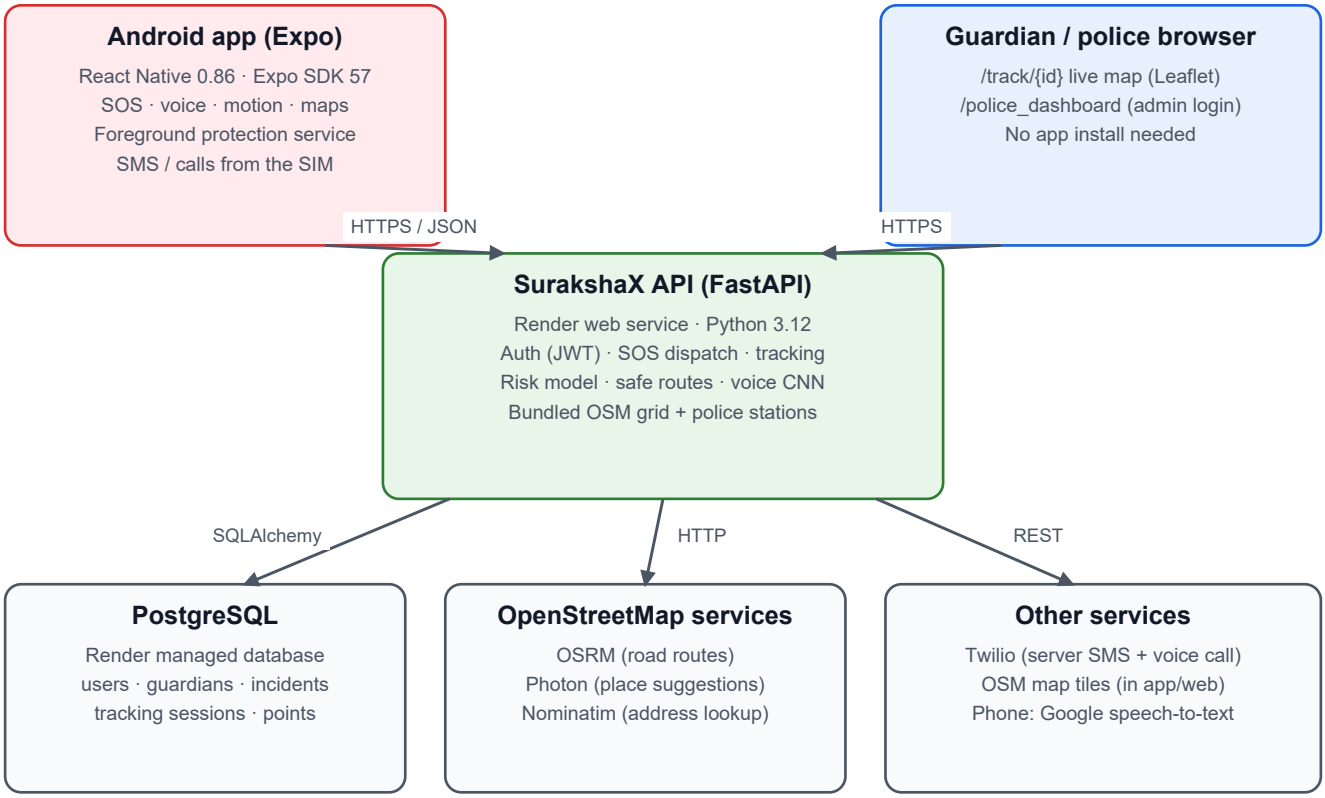


Figure 1. Components and how they talk to each other.

SOS sequence

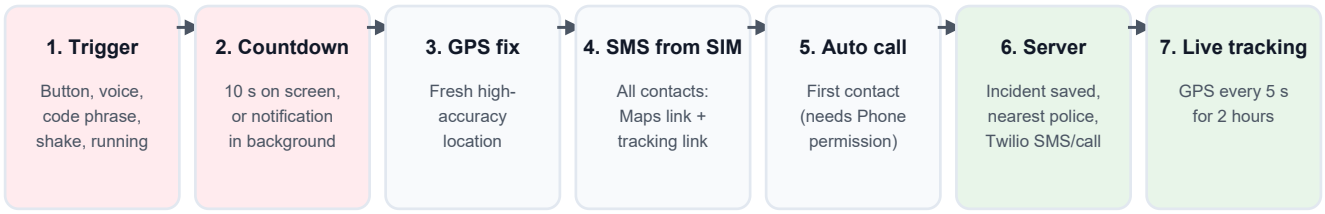


Figure 2. What happens after an SOS is triggered. The SMS goes out from the SIM first, so it works even without internet; server steps follow when online.

Repository layout

Folder	What it holds
`mobile/`	Expo / React Native app: screens, contexts (state), components, utils, native module in modules/surakshax-native
`server/`	FastAPI backend (app/), bundled models (models/), map data (data/), tests, grid build script
`render.yaml`	Render Blueprint: web service + PostgreSQL database
`backend/`	Previous Flask backend; its ml/ folder holds the training scripts and evaluation reports
`app/`	Legacy native Java Android app (not maintained)
`docs/`	This report and its generator script

3. Tech stack

Mobile app

Area	Technology	Version / notes
Framework	React Native + Expo (managed workflow, Continuous Native Generation)	RN 0.86.3 · Expo SDK 57 · React 19.2
Language	JavaScript (app) · Kotlin (native module)	Hermes JS engine
Navigation	React Navigation: native stack + bottom tabs; custom side menu	v7
Location	expo-location (GPS fixes, watchPosition, reverse geocoding fallback)	~57.0
Speech-to-text	expo-speech-recognition (Android SpeechRecognizer / Web Speech API)	57.1.1
Audio	expo-audio (3 s clips for the server voice model)	~57.0
Sensors	expo-sensors Accelerometer (shake / running)	~57.0
Maps	Leaflet 1.9.4 in react-native-webview, OpenStreetMap tiles	WebView 13.16
Storage	@react-native-async-storage (profile, contacts, history, settings)	2.2
UI	Custom components (AppHeader, Card, ListRow…), @expo/vector-icons Material icons	crimson theme
Native module	surakshax-native (Kotlin): SIM SMS, direct calls, foreground protection service, native SOS countdown notification, permission status	Expo Modules API
Build	Gradle 9.3 · Android SDK 36 · NDK 27.1 · min Android 7 (API 24)	release APK, debug-key signed

Backend

Area	Technology	Version / notes
Language / runtime	Python	3.12 on Render (3.13 locally)
Web framework	FastAPI + Uvicorn (ASGI)	0.115 · 0.34; OpenAPI docs at /docs
Database access	SQLAlchemy 2 ORM + psycopg 3	PostgreSQL on Render; SQLite file locally
Auth	PBKDF2-SHA256 password hashing (260,000 rounds) + JWT (PyJWT, HS256)	tokens valid 180 days
ML inference	ONNX Runtime (voice CNN) · scikit-learn + joblib (Random Forest) · NumPy · SciPy	no PyTorch at runtime
Audio decoding	PyAV (FFmpeg) for m4a / webm / wav uploads	
HTTP clients	requests (+ truststore for corporate TLS proxies)	OSRM, Photon, Nominatim, Overpass
Messaging	Twilio Python SDK (optional SMS + voice call)	
Tests	pytest + FastAPI TestClient	14 tests

ML training (offline, not deployed)

Purpose	Tools
Voice dataset	edge-tts (Microsoft neural TTS, 43 voices, 11 Indian languages/accents), librosa, soundfile, augmentation (noise, reverb, phone band, AAC)
Voice model training	PyTorch (CNN), exported to ONNX opset 17; MFCC front-end in pure NumPy shared with serving

Purpose	Tools
Risk model	scikit-learn RandomForestClassifier on a documented synthetic dataset
Map grid	pyosmium over the Geofabrik India OpenStreetMap extract (scripts/build_area_grid.py)

Infrastructure

Piece	Choice
Hosting	Render Blueprint (render.yaml): free web service + free PostgreSQL
Source control	Git, GitHub repository Jotthecode/Suraksha_X
Mobile distribution	Sideloaded release APK (66 MB, arm64-v8a / armeabi-v7a / x86_64)

4. Features

Feature	What it does	Built with
Account & sign-up	4-step sign-up (account, about you, emergency contacts, review) with validation; login; profile sync	FastAPI auth, JWT
App walkthrough	5-slide tour on first login; replay from the menu or Settings	React Native ScrollView paging
SOS button	10-second cancellable countdown, then the full SOS sequence (Figure 2)	SIM SMS, Telecom call, API
Voice SOS	Hears "Bachao", "Help me", "Save me" (Hindi script too) and repeated "help"	Speech-to-text; ONNX CNN fallback
Secret code phrases	Up to 5 everyday sentences; saying one sends a silent SOS (no screen, sound or vibration)	On-phone phrase matching
Shake to SOS	4 hard shakes (≥2.7 g) within 2 s	Accelerometer
Sudden running	≥150 steps/min with hard strikes for ~8 s, and ≥8 km/h when GPS speed is known	Accelerometer + GPS
Background protection	Foreground service keeps voice, motion and location working in other apps / screen off; countdown becomes a notification with "I'm safe"	Kotlin service, wake lock
Auto call	Rings the first contact right after the SOS text	TelecomManager.placeCall
Live location sharing	Unguessable link; guardians watch a live Leaflet map; 2-hour sessions	watchPosition → API → /track
Area safety	Low / Medium / High risk for the current place and time	Random Forest + OSM grid
Safe route	Compares OSRM road alternatives by distance and risk; Safest / Balanced / Faster / Shortest	OSRM + risk sampling
Place search	Suggestions as you type, nearest first, anywhere in India	Photon (OSM)
Nearby police	Nearest stations with distance, call and directions	Bundled OSM dataset (4,370)
Emergency numbers	One-tap dialer for 112, 1091 and the first contact	tel: links
SOS history & recordings	Local incident log; audio evidence recorder and player	AsyncStorage, expo-audio
Permissions check	Shows missing SMS/Phone/Mic/Location/Notification/Battery settings and opens the right screen	Kotlin module
Police dashboard	Password-protected web list of SOS alerts with live-map links	Server-rendered HTML
Testing tools	Set a test location, try keywords, sensor check	In-app screen

5. How the backend works

The API is a single FastAPI application (server/app/main.py) with four routers. On start-up it creates the database tables if needed and loads the two ML models in a background thread so the first request is fast. Every error is returned as JSON `{"success": false, "error": "…"}`, which the app shows to the user directly.

Module	Responsibility
`routers/auth.py`	Register (full validation: email, 10-digit phone, gender, blood group, ≥1 contact), login, profile, contacts, delete account
`routers/safety.py`	Public read-only tools: area risk, hotspot grid, safe route, geocode, place suggestions, reverse lookup, nearest police, voice detection, sensor check
`routers/sos.py`	SOS dispatch, live-tracking updates/stop, public tracking JSON and map page, the user's incident history
`routers/admin.py`	Police / admin dashboard and incidents JSON (HTTP Basic auth)
`services/risk.py`	Random Forest risk model; features from the bundled grid, police distance from the station dataset
`services/routing.py`	Safe-route planner over OSRM alternatives
`services/area_grid.py`	Reads data/india_area_grid.npz: lamps within 250 m, shops/amenities within 400 m
`services/geo.py`	OSRM, Photon, Nominatim, Overpass clients with caching, time limits and fallbacks
`services/voice.py`	MFCC + ONNX CNN distress-word detection; text keyword fallback
`services/notify.py`	Twilio SMS to every contact + voice call to the first contact
`pages.py`	HTML for the guardian tracking page and the police dashboard
`security.py`	Password hashing, JWT creation/validation, admin check

SOS dispatch on the server

- Requires a logged-in user (Bearer token). Creates or reuses the tracking session named by the app (128-bit random id), stores the first location point and builds the public tracking URL.
- Contacts come from the request, or the user's saved contacts. The two nearest police stations are found with the Haversine formula over the bundled dataset.
- Twilio is used only if configured, and at most 5 server-side sends per user per 10 minutes, so a stolen token can't run up a bill. Every SOS is still recorded as an incident.

Area risk and safe routes

For a point, the server reads street lamps and shops/amenities around it from the bundled grid, the distance to the nearest police station, and the current time and weekday, then asks the Random Forest for Low/Medium/High probabilities. The risk score is 0.18·P(low) + 0.50·P(medium) + 0.85·P(high). For a route, OSRM supplies real road alternatives (walking first, driving fallback) plus two detours through points left and right of the direct line; each is sampled every ~150 m and scored:

`weight = α · (length / longest length) + (1 − α) · mean risk` with α = 0.2 (Safest) … 0.8 (Shortest), default 0.4.

The route with the lowest weight is the safe route; the shortest is shown alongside. Typical response: about 1 s (3–5 s on a cold server).

Performance safeguards: map data comes from the bundled grid (public Overpass servers are often overloaded and are only a fallback, with a 4 s limit and 3-minute cool-down); routing times out after 10 s; suggestions and reverse lookups are cached; Nominatim is limited to 1 request per second.

6. API reference

Base URL: https://surakshax-api.onrender.com · interactive docs at /docs. Auth: **Bearer** = JWT from register/login; **Admin** = HTTP Basic (ADMIN_USERNAME / ADMIN_PASSWORD); **Link** = anyone with the tracking id.

Method	Path	Auth	Purpose
GET	`/health`	–	Status, version, database check, Twilio on/off
POST	`/api/register`	–	Create account with profile and contacts → {token, user}
POST	`/api/login`	–	Email + password → {token, user}
GET / PUT	`/api/user/profile`	Bearer	Read / update name, phone, gender, blood group, DOB, address, medical notes
POST	`/api/user/guardians`	Bearer	Replace emergency contacts (max 10)
DELETE	`/api/user`	Bearer	Delete the account and its data
GET	`/api/incidents`	Bearer	The user's last 50 SOS incidents
POST	`/emergency_dispatch`	Bearer	Record SOS, nearest police, optional Twilio SMS + call
POST	`/tracking/update`	Bearer	Add a location point to the user's tracking session
POST	`/tracking/{id}/stop`	Bearer	End location sharing
GET	`/tracking/{id}`	Link	Live location JSON (latest point + up to 500-point trail)
GET	`/track/{id}`	Link	Guardian map page (Leaflet, refreshes every 5 s)
POST	`/predict_risk`	–	Area risk for a point
GET	`/hotspots`	–	Risk grid around a point (map overlay)
POST	`/safe_route`	–	Safe vs shortest route with comparison metrics
GET	`/places/suggest`	–	Type-ahead place suggestions (Photon), nearest first
GET	`/places/reverse`	–	Short address for coordinates (Nominatim)
GET	`/geocode`	–	Full address search (Nominatim)
POST	`/police/nearest`	–	Nearest police stations (1–20)
POST	`/voice_sos`	–	Distress-word detection: audio upload, base64 audio, or text
POST	`/analyze_status`	–	Rule-based check of scream score, fall force, heart rate
GET	`/police_dashboard`	Admin	SOS alerts web dashboard (auto-refresh 20 s)
GET	`/admin/incidents`	Admin	All incidents as JSON

Example: register

```
POST /api/register
{ "email": "asha@example.com", "password": "••••••", "name": "Asha",
  "phone": "9876543210", "gender": "Female", "blood_group": "B+",
  "guardians": [{ "name": "Mom", "phone": "9876500001", "relation": "Mother" }] }
→ 200 { "success": true, "token": "eyJ…", "user": { … } }
→ 400 { "success": false, "error": "Add at least one emergency contact" }
```

7. Database schema

PostgreSQL in production (SQLite locally), defined with SQLAlchemy 2 in server/app/models.py. Tables are created automatically on start-up. Deleting a user cascades to all of their rows.

users

One row per account.

Column	Type	Notes
id	VARCHAR(32) PK	Random hex id
email	VARCHAR(255) UNIQUE	Lower-cased; indexed
password_hash	VARCHAR(255)	pbkdf2_sha256$260000$salt$hash
name / phone	VARCHAR(120) / (32)	Phone validated as 10-digit Indian mobile
gender / blood_group	VARCHAR(32) / (8)	Fixed option sets
date_of_birth	VARCHAR(10)	YYYY-MM-DD, optional
address / medical_notes	VARCHAR(300) / (500)	Optional
created_at	TIMESTAMPTZ	

guardians

Emergency contacts, ordered.

Column	Type	Notes
id	INTEGER PK	Auto-increment
user_id	FK → users.id	ON DELETE CASCADE; indexed
position	INTEGER	Order in the list (first = called automatically)
name / phone / relation	VARCHAR	Relation defaults to Guardian
extra	JSON	Any other fields the app stores, round-tripped

tracking_sessions

One live-location share (SOS or manual).

Column	Type	Notes
id	VARCHAR(64) PK	128-bit random id = the secret in the link
user_id	FK → users.id	Only the owner can push points
user_name	VARCHAR(120)	Shown on the map page
created_at / expires_at	TIMESTAMPTZ	Expires after 2 hours
is_active / is_emergency	BOOLEAN	
last_lat / last_lon / last_accuracy / last_speed	FLOAT	Latest point, for fast reads
last_battery / last_update	VARCHAR(16) / TIMESTAMPTZ	

location_points

Trail of a tracking session.

Column	Type	Notes
id	INTEGER PK	
session_id	FK → tracking_sessions.id	ON DELETE CASCADE; indexed
lat / lon	FLOAT	
recorded_at	TIMESTAMPTZ	

incidents

Every SOS sent to the server.

Column	Type	Notes
id	INTEGER PK	Shown as INC-000123
user_id	FK → users.id	Indexed
session_id	VARCHAR(64)	Tracking session of this SOS
user_name / trigger	VARCHAR	e.g. "Voice code phrase", "Phone shaken hard"
lat / lon	FLOAT	Location at dispatch
nearest_stations / contacts / sms_status	JSON	Snapshots at dispatch time
call_status	JSON	Twilio call result
status / notes	VARCHAR / TEXT	ACTIVE by default
created_at	TIMESTAMPTZ	Indexed (rate limit + dashboard)

Relationships: users 1—n guardians · users 1—n tracking_sessions 1—n location_points · users 1—n incidents.
On the phone (AsyncStorage): profile, contacts (offline copy), SOS history (last 50), recordings list, trigger settings and code phrases. Code phrases never leave the phone.

8. Machine-learning models

8.1 Voice-distress keyword CNN

Recognises distress words in 1-second audio windows on the server (/voice_sos). Used by the app when the phone's speech-to-text service is unavailable; otherwise the phone's own speech recognition is used.

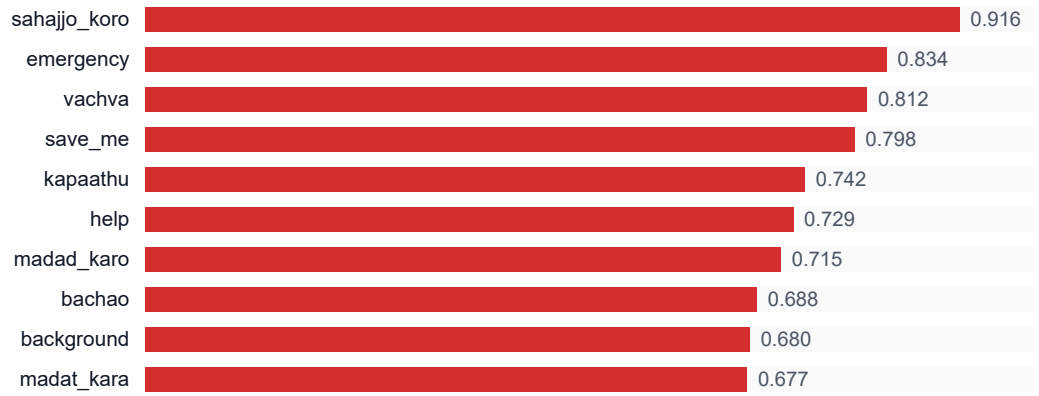
Item	Detail
Task	10 classes: help, save_me, emergency, bachao, madad_karo, madat_kara, vachva, kapaathu, sahajjo_koro, background
Languages	English, Hindi, Marathi, Tamil, Bengali (keywords); 43 voices across 11 Indian languages/accents
Data	8,039 synthetic 1 s clips at 16 kHz (Microsoft neural TTS via edge-tts) + augmentation: noise, babble, reverb, phone band, AAC, speed/pitch. Background class 1,923 clips
Splits	train 5,623 · val 935 · test_seen 929 · test_heldout 552 (3 speakers never seen in training)
Features	40 MFCC × 101 frames (25 ms Hamming, 10 ms hop), per-clip normalisation, pure NumPy
Architecture	2 × [Conv2D + BatchNorm + ReLU + MaxPool] → FC(128) → FC(10), dropout 0.2/0.3; 839,626 parameters
Training	PyTorch, AdamW (wd 1e-3), warm-up + cosine LR, SpecAugment, class-weighted loss, early stopping; 51 epochs (best 36), 40 min on CPU
Deployment	ONNX opset 17 (3.36 MB) on ONNX Runtime; matches PyTorch to 5.7e-6
Decision rule	SOS if max non-background probability ≥ 0.85; the app also requires P ≥ 0.95 once or two windows in a row
Latency	8.06 ms per 1 s window on CPU (2.44 ms model + 5.62 ms MFCC); 56 ms for a 3 s clip

Split	n	Accuracy	Macro F1	Distress precision	Distress recall	False-alarm rate
val	935	0.735	0.754	0.973	0.509	4.4%
test_seen	929	0.744	0.752	0.982	0.469	2.8%
test_heldout (new speakers)	552	0.754	0.770	0.980	0.553	4.2%
test_all	1,481	0.748	0.759	0.981	0.501	3.3%

Distress metrics at the 0.85 threshold. The model is tuned to avoid false alarms (98% precision) at the cost of missing about half of single windows, which is why the phone's speech-to-text is the primary engine.

Caveat: trained and tested on synthetic speech. Real panicked speech, children's voices and real phone microphones will score lower. Adding real recordings to backend/ml/data/voice_real/ and retraining is the single most useful improvement (see backend/ml/README.md).

Per-class F1 (test_all)



8.2 Area crime-risk Random Forest

Item	Detail
Task	Classify a place at a time as Low / Medium / High risk; also gives a 0–1 risk score
Features (5)	time_of_day_bin (6 four-hour bins) · day_of_week · street_lighting (0–1) · activity_density (0–1) · police_distance_km (0–15)
Feature source at runtime	Lighting = lamps within 250 m / 8 (or 0.85 × activity on busy streets); activity = shops+amenities within 400 m / 40; both from the bundled OSM grid. Police distance from 4,370 stations
Training data	20,000 synthetic samples labelled by a documented heuristic (night, darkness, isolation, distance from police raise risk); not real crime records
Model	scikit-learn RandomForestClassifier: 100 trees, max depth 12, min samples split 4, balanced class weights
Evaluation	Stratified 80/20 split (16,000 / 4,000); deployed model refit on all 20,000

Metric	Value	Class	Precision	Recall	F1	Support
Accuracy	0.875	Low Risk	0.907	0.893	0.900	1,918
Weighted F1	0.875	Medium Risk	0.794	0.813	0.803	1,256
Macro F1	0.877	High Risk	0.929	0.929	0.929	826
Macro AUC-ROC (OvR)	0.970					



Left: confusion matrix on the 4,000 test samples (rows true, columns predicted). Right: feature importances.

Caveat: the metrics show how well the forest learns the synthetic risk rule, not real-world crime. Replacing generate_training_data() with geocoded incident data (e.g. NCRB / FIR records) is needed before the score can be called evidence-based.

8.3 Bundled OpenStreetMap feature grid

Item	Detail
Source	Geofabrik India OpenStreetMap extract (20 Oct 2026, 1.71 GB), © OpenStreetMap contributors (ODbL)
Content	29,288 street lamps and 485,667 shops/amenities in 287,948 non-empty 0.001° cells (~110 × 105 m)
Size / speed	0.7 MB file; lookups in ~1 ms; no network needed
Refresh	python server/scripts/build_area_grid.py india-latest.osm.pbf (about 1 minute)

9. Rule-based detection on the phone

These run on the phone without ML models and work while the app is in the background (Android foreground service).

Detector	Rule	Guards against false alarms
Code phrase	Normalised transcript contains the saved phrase as whole words (min 2 words)	Phrases are personal and unusual; 2-word minimum
Strong distress words	bachao (and spelling variants), बचाओ, help me, save me, please help, madad karo, मदद करो	Whole-word matching ("helpful" ≠ "help")
Weak words	help, emergency, madad	Must be repeated within 6 s, only on final transcripts
Shake	4 jolts ≥ 2.7 g within 2 s (accelerometer at 20 Hz)	Single bumps and drops ignored
Sudden running	≥150 steps/min over 8 s, median foot strike ≥1.6 g, GPS speed ≥8 km/h when known	Walking (~110 steps/min) and running in place ignored
Cooldown	One automatic SOS per minute	Prevents repeated alerts

Speech recognition fallback chain

- Google speech service (en-IN, continuous) → Google basic (en-US) → phone default service → on-device recognition → server voice CNN (distress words only).
- A setup that fails twice to start (8-second watchdog) is skipped; the real error is shown on the Safety triggers screen with a live "Heard:" line for testing.

10. External services and data

Service	Used for	Key / cost
OSRM (routing.openstreetmap.de, router.project-osrm.org)	Walking / driving road routes	Free, no key; demo servers
Photon (photon.komoot.io)	Place suggestions while typing	Free, no key
Nominatim (nominatim.openstreetmap.org)	Address search, reverse lookup	Free, no key; 1 req/s
Overpass API	Fallback map data if the grid is missing	Free, no key; often overloaded
OpenStreetMap tiles	Map images in the app and tracking page	Free, attribution required
Leaflet 1.9.4 (unpkg CDN)	Map rendering	Free
Twilio	Server SMS + voice call (optional)	Paid per message / minute
Google speech services (on the phone)	Speech-to-text for voice SOS	Free, part of Android
Render	Hosting the API and PostgreSQL	Free tier (sleeps after 15 min)

No Google Maps or other paid map API key is needed. For many users, the OSM services' usage policies ask apps to move to a paid or self-hosted provider (e.g. MapTiler / Stadia tiles, a self-hosted OSRM).

11. Security and privacy

- Passwords: PBKDF2-SHA256, 260,000 iterations, random 16-byte salt; compared in constant time.
- Sessions: JWT (HS256) signed with a server secret generated by Render; 180-day expiry.
- SOS dispatch and tracking updates require login; a user can only write to their own tracking session.
- Tracking links use 128-bit random ids; sessions expire after 2 hours and can be stopped.
- Police dashboard behind HTTP Basic auth; disabled until ADMIN_PASSWORD is set.
- Server SMS/call rate limit: 5 per user per 10 minutes.
- Secrets (.env, local database, Firebase config, old user file) are excluded from the Git repository.
- Code phrases stay on the phone; the SMS says "code phrase", never the phrase itself.
- Silent SOS shows nothing on screen, so a code phrase can be used in front of an attacker.

Android permissions used

Permission	Why
`SEND_SMS · CALL_PHONE`	Send SOS texts and call the first contact without a tap. Android 13+ blocks these for APK installs until "Allow restricted settings" is enabled in App info.
`RECORD_AUDIO`	Voice SOS and evidence recordings
`ACCESS_FINE / COARSE_LOCATION`	SOS location, live tracking, area risk, routes
`FOREGROUND_SERVICE (+ MICROPHONE / LOCATION / SPECIAL_USE)`	Background protection
`POST_NOTIFICATIONS · WAKE_LOCK`	Protection and SOS countdown notifications; keep detecting with the screen off
`INTERNET · VIBRATE`	API calls; SOS haptics

12. Deployment and configuration

Variable	Purpose
`DATABASE_URL`	PostgreSQL connection (wired by render.yaml); empty = local SQLite
`JWT_SECRET`	Signs login tokens (generated by Render)
`ADMIN_USERNAME / ADMIN_PASSWORD`	Police dashboard login
`PUBLIC_BASE_URL`	Base of tracking links in SMS
`OSM_USER_AGENT`	Identifies the app to OpenStreetMap services
`TWILIO_ACCOUNT_SID / AUTH_TOKEN / PHONE_NUMBER`	Optional server SMS + call
`SOS_MAX_PER_WINDOW / SOS_WINDOW_MINUTES`	Twilio rate limit (default 5 per 10 min)
`TRACKING_TTL_HOURS · CORS_ORIGINS · OVERPASS_BUDGET_S`	Tracking lifetime, browser origins, Overpass time limit
`EXPO_PUBLIC_API_URL (app build)`	API address built into the APK

- Deploy: push to GitHub → Render Blueprint creates the web service and database from render.yaml.
- Build the APK: npx expo prebuild -p android, then gradlew assembleRelease (EXPO_PUBLIC_API_URL set).
- Tests: python -m pytest in server/ (14 tests, SQLite, no network).

13. Limitations and next steps

Area	Current state	Next step
Voice CNN	Synthetic training data; ~50% single-window recall	Collect real recordings (with consent) and retrain
Risk model	Synthetic labels	Train on geocoded incident data
Background	Works via foreground service; some brands (Xiaomi, Oppo, Vivo, Samsung) kill background apps	Ask users to set Battery to Unrestricted; test per brand
Speech in background	Depends on the phone's speech service allowing background use	Test on target phones; on-device model
Hosting	Render free tier sleeps; free PostgreSQL may expire	Paid instance and database before launch
Map services	Public OSM demo servers	Paid / self-hosted tiles and routing at scale
Distribution	Debug-key signed APK	Release keystore or EAS build for the Play Store (SMS/Call policy review)
iOS	Android only (native module is Android)	iOS port of SMS/call/background features
Schema changes	Tables created automatically, no migrations	Add Alembic migrations